Lab 7 Fuzz Testing Evidence

Water Utility Billing Input Validation

The starter application accepted the meter ID, then stopped on `10,,30` because the empty CSV field reached integer conversion.

Buggy application output

```
PS C:\water-utility-assignment\Lab 7> python processor.py
==== Running sanitize_meter_id ====
Enter a meter ID with prefixes (!,@,#,$,%): Sanitized Meter ID: METER01

==== Running parse_consumption_readings ====
Enter consumption readings (e.g. 10,25,30): Traceback (most recent call last):
  File "C:\Users\carbo\projects\caliper\water-utility-assignment\Lab 7\processor.py", line 35, in <module>
    main()
  File "C:\Users\carbo\projects\caliper\water-utility-assignment\Lab 7\processor.py", line 26, in main
    readings_list = parse_consumption_readings(csv_input)
                    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\carbo\projects\caliper\water-utility-assignment\Lab 7\processor.py", line 9, in parse_consumption_readings
    return [int(r) for r in readings]
            ^^^^^^
ValueError: invalid literal for int() with base 10: ''
```

Hypothesis found input failures

The initial fuzz run failed all three tests. None caused attribute errors in each function; malformed billing records could also cause an index error. The image shows the failure summary excerpt; the full captured output is included alongside this PDF.

Hypothesis failures

```
PS C:\water-utility-assignment\Lab 7> python -m pytest -q test_processor.py
FFF                                                                      [100%]
E       AttributeError: 'NoneType' object has no attribute 'strip'
E       AttributeError: 'NoneType' object has no attribute 'split'
    | IndexError: list index out of range
    | AttributeError: 'NoneType' object has no attribute 'split'
FAILED test_processor.py::test_sanitize_meter_id_no_crash - AttributeError: '...
FAILED test_processor.py::test_parse_consumption_readings_safe - AttributeErr...
FAILED test_processor.py::test_format_billing_summary_safe - ExceptionGroup: ...
3 failed in 2.22s
```

Fixed application with valid inputs

The application now validates and processes a normal meter ID, comma-separated readings, and a `Name:Usage:Zone` record.

Fixed application output

```
PS C:\water-utility-assignment\Lab 7> python processor.py
==== Running sanitize_meter_id ====
Enter a meter ID with prefixes (!,@,#,$,%): Sanitized Meter ID: MTR1234

==== Running parse_consumption_readings ====
Enter consumption readings (e.g. 10,25,30): Parsed Readings: [10, 25, 30]

==== Running format_billing_summary ====
Enter account record (Name:Usage:Zone): Billing Summary: Customer Alice billed 100 USD in North
```

Invalid inputs are reported safely

Malformed values now produce field-specific messages. The application continues to the next field and exits without an unhandled traceback.

Invalid inputs handled

```
PS C:\water-utility-assignment\Lab 7> python processor.py
==== Running sanitize_meter_id ====
Enter a meter ID with prefixes (!,@,#,$,%): Invalid meter ID: Meter ID cannot be empty

==== Running parse_consumption_readings ====
Enter consumption readings (e.g. 10,25,30): Invalid consumption readings: Readings cannot contain empty values

==== Running format_billing_summary ====
Enter account record (Name:Usage:Zone): Invalid account record: Account record must use Name:Usage:Zone format
```

Hypothesis tests pass after the fix

The three property-based tests complete successfully after the input checks were added.

Hypothesis passing output

```
PS C:\water-utility-assignment\Lab 7> python -m pytest -q test_processor.py
...                                                                      [100%]
3 passed in 1.61s
```